Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

MINISTÉRIO DA DEFESA
EXÉRCITO BRASILEIRO
DEPARTAMENTO DE CIÊNCIA E TECNOLOGIA
INSTITUTO MILITAR DE ENGENHARIA
CURSO DE GRADUAÇÃO EM ENGENHARIA DA COMPUTAÇÃO

JOÃO VITOR CAMELO DE ALMEIDA
THIAGO CALHEIROS DE SOUZA BARBOSA

PLANEJAMENTO DE TRAJETÓRIA PARA ROBÔ TERRESTRE, UTILIZANDO OTIMIZAÇÃO EM GRAFOS

RIO DE JANEIRO
2025

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

JOÃO VITOR CAMELO DE ALMEIDA
THIAGO CALHEIROS DE SOUZA BARBOSA

PLANEJAMENTO DE TRAJETÓRIA PARA ROBÔ TERRESTRE, UTILIZANDO OTIMIZAÇÃO EM GRAFOS

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Claudia Marcela Justel, D.Sc.
Paulo Fernando Ferreira Rosa, Ph.D.

Rio de Janeiro

2025

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

©2025
INSTITUTO MILITAR DE ENGENHARIA
Praça General Tibúrcio, 80 – Praia Vermelha
Rio de Janeiro – RJ CEP: 22290-270

Este exemplar é de propriedade do Instituto Militar de Engenharia, que poderá incluí-lo em base de dados, armazenar em computador, microfilmar ou adotar qualquer forma de arquivamento.

É permitida a menção, reprodução parcial ou integral e a transmissão entre bibliotecas deste trabalho, sem modificação de seu texto, em qualquer meio que esteja ou venha a ser fixado, para pesquisa acadêmica, comentários e citações, desde que sem finalidade comercial e que seja feita a referência bibliográfica completa.

Os conceitos expressos neste trabalho são de responsabilidade do(s) autor(es) e do(s) orientador(es).

Camelo de Almeida, João Vitor; Calheiros de Souza Barbosa, Thiago.
Planejamento de trajetória para robô terrestre, utilizando otimização em grafos / João Vitor Camelo de Almeida e Thiago Calheiros de Souza Barbosa. – Rio de Janeiro, 2025.
47 f.

Orientador(es): Claudia Marcela Justel e Paulo Fernando Ferreira Rosa.

Projeto de Final de Curso (graduação) – Instituto Militar de Engenharia, Engenharia da Computação, 2025.

1. Veículos terrestres não tripulados. 2. Algoritmos baseados em grafos. 3. Planejamento de trajetórias. 4. Processamento de imagens aéreas. i. Justel, Claudia Marcela (orient.) ii. Ferreira Rosa, Paulo Fernando (orient.) iii. Título

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

JOÃO VITOR CAMELO DE ALMEIDA
THIAGO CALHEIROS DE SOUZA BARBOSA

Planejamento de trajetória para robô terrestre, utilizando otimização em grafos

Projeto de Final de Curso apresentado ao Curso de Graduação em Engenharia da Computação do Instituto Militar de Engenharia, como requisito parcial para a obtenção do título de Bacharel em Engenharia da Computação.

Orientador(es): Claudia Marcela Justel e Paulo Fernando Ferreira Rosa.

Aprovada em 13 de outubro de 2025, pela seguinte banca examinadora:

Assinado por: Paulo Fernando Ferreira Rosa
FC5567F32C85404...
Prof. **Paulo Fernando Ferreira Rosa** - Professor do IME - Presidente

Signed by: Claudia Marcela Justel
EF042FD4D9734CA...
Profa. **Claudia Marcela Justel** - Professora do IME

Assinado por: Luiz Henrique Da Costa Araujo
2BEF3F8B01AD446...
Prof. **Luiz Henrique Da Costa Araujo** - Professor do IME

Assinado por: Oscar Martins Wanderley Filho
05297D77346649E...
Cap **Oscar Martins Wanderley Filho** - Professor do IME

Assinado por: Diogo Pimentel Pinheiro
E6E1F1389C0D424...
Maj **Diogo Pimentel Pinheiro** - Representante do CTEx

Rio de Janeiro
2025

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Às nossas famílias.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

AGRADECIMENTOS

Os agradecimentos principais são direcionados aos nossos orientadores por todo acompanhamento e ensinamentos.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

"Não vos amoldeis às estruturas deste mundo,
mas transformai-vos pela renovação da mente,
a fim de distinguir qual é a vontade de Deus:
o que é bom, o que Lhe é agradável, o que é perfeito.
(Bíblia Sagrada, Romanos 12, 2)

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

RESUMO

Este projeto tem como objetivo o desenvolvimento e avaliação de algoritmos baseados em grafos aplicados ao planejamento de trajetória de veículos terrestres não tripulados. A proposta parte da necessidade de sistemas de navegação mais precisos e eficientes, capazes de lidar com ambientes complexos e restrições reais de deslocamento. Para isso, é apresentada uma nova metodologia de processamento de imagens aéreas, utilizando segmentação semântica para identificação das regiões do terreno e posterior conversão dessas informações em representações na forma de grafos. Sobre tais estruturas foram aplicados e avaliados os algoritmos clássicos de Dijkstra e A*, amplamente reconhecidos por sua capacidade de encontrar caminhos ótimos. Os resultados obtidos foram comparados com a execução do algoritmo randômico Rapidly-exploring Random Trees (RRT), bastante utilizado em cenários de exploração.

As análises demonstraram que os algoritmos baseados em grafos apresentaram desempenho mais consistente, confiável e previsível em relação ao RRT, destacando-se pela característica de determinismo e pela garantia de otimalidade das soluções encontradas. O RRT, por sua vez, embora apresente flexibilidade e rapidez em alguns casos, mostrou maior variabilidade nos resultados devido ao seu caráter não determinístico. Em conjunto, os experimentos indicam que a abordagem proposta, ao integrar segmentação semântica de imagens e planejamento baseado em grafos, constitui uma alternativa promissora para aplicações em veículos autônomos, contribuindo para maior segurança e eficiência em cenários de navegação terrestre.

Palavras-chave: Veículos terrestres não tripulados; Algoritmos baseados em grafos; Planejamento de trajetórias; Processamento de imagens aéreas.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

ABSTRACT

This project aims to develop and evaluate graph-based algorithms applied to trajectory planning of unmanned ground vehicles. The proposal arises from the need for more precise and efficient navigation systems capable of handling complex environments and real-world motion constraints. To this end, a new methodology for processing aerial images is presented, employing semantic segmentation to identify terrain regions and subsequently converting this information into graph representations. On these structures, the classical Dijkstra and A* algorithms—widely recognized for their ability to compute optimal paths—were applied and assessed. The results were compared with the execution of the Rapidly-exploring Random Trees (RRT) algorithm, commonly used in exploration scenarios.

The analyses demonstrated that the graph-based algorithms exhibited more consistent, reliable, and predictable performance when compared to RRT, mainly due to their deterministic nature and the guarantee of optimality in the solutions obtained. RRT, in contrast, although flexible and sometimes faster, showed greater variability in the results due to its non-deterministic behavior. Overall, the experiments indicate that the proposed approach, by integrating semantic segmentation of images with graph-based planning, constitutes a promising alternative for autonomous vehicle applications, contributing to greater safety and efficiency in ground navigation scenarios.

Keywords: Unmanned ground vehicles; Graph-based algorithms; Trajectory planning; Aerial image processing.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

LISTA DE ILUSTRAÇÕES

Figura 1 – Histograma de distribuição de trabalhos por ano 24
Figura 2 – Distribuição de trabalhos por país 25
Figura 3 – Frequência de aparições de palavras chave 26
Figura 4 – A imagem retirada de (1) ilustra um robô aéreo sobrevoando o terreno da missão. Os triângulos verdes representam um conjunto de robôs terrestres e o círculo vermelho representa o alvo da missão. A operação é controlada por um operador na estação de controle em solo. 29
Figura 5 – As imagens segmentadas obtidas utilizando as cinco arquiteturas de redes diferentes em (1). 30
Figura 6 – Imagens exemplificando a criação dos grafos. Da esquerda para a direita, de cima para baixo: imagem segmentada, divisão em pixeis, máscara binária, vértices e arestas, pesos no vértice, arestas direcionadas com peso (vermelho indica peso 1, azul peso 2). 35
Figura 7 – As 5 imagens aéreas utilizadas para as análises (Imagens 1 a 5, da esquerda para a direita, de cima para baixo) 36
Figura 8 – As correspondentes imagens segmentadas. Na primeira imagem, a cor roxo escuro representa as áreas transitáveis, enquanto, no restante, a cor azul possui essa função. 37
Figura 9 – Imagens com aplicação da máscara binária. 37
Figura 10 – Caminhos encontrados ao aplicar o A* nas imagens 40
Figura 11 – Caminhos encontrados ao aplicar o algoritmo de Dijkstra nas imagens . 41
Figura 12 – Caminhos encontrados ao aplicar o RRT nas imagens 42

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

LISTA DE QUADROS

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

LISTA DE TABELAS

Tabela 1 – Comparação de algoritmos de planejamento de trajetória baseados em diferentes abordagens. 28
Tabela 2 – Resultados do algoritmo A* 38
Tabela 3 – Resultados do algoritmo de Dijkstra 38
Tabela 4 – Resultados do algoritmo RRT 39

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

LISTA DE ABREVIATURAS E SIGLAS

BFS	Breadth-First Search
CTEx	Centro Tecnológico do Exército
EB	Exército Brasileiro
IA	Inteligência Artificial
RRT	Rapidly-exploring Random Tree
SVTRP	Sistemas de Veículos Terrestres Remotamente Pilotados
UAV	Unmanned Aerial Vehicle
UGV	Unmanned Ground Vehicle

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

SUMÁRIO

1 INTRODUÇÃO 14
1.1 MOTIVAÇÃO E JUSTIFICATIVA 15
1.2 OBJETIVOS 15
1.3 METODOLOGIA 15
1.4 ESTRUTURA 16

2 CONCEITOS PRELIMINARES 17
2.1 VEÍCULOS TERRESTRES AUTÔNOMOS 17
2.2 PLANEJAMENTO DE CAMINHOS 17
2.3 GRAFOS 18
2.4 ALGORITMOS DE CAMINHOS MÍNIMOS EM GRAFOS 18
2.4.1 BREADTH-FIRST SEARCH (BFS) 19
2.4.2 ALGORITMO DE DIJKSTRA 20
2.4.3 ALGORITMO A* 21
2.5 RAPIDLY-EXPLORING RANDOM TREE (RRT) 23

3 REVISÃO DE LITERATURA 24
3.1 REVISÃO BIBLIOMÉTRICA 24
3.2 REVISÃO BIBLIOGRÁFICA 26

4 METODOLOGIA 31
4.1 COLETA DOS DADOS (MAPEAMENTO) 31
4.2 CONSTRUÇÃO DOS GRAFOS 31
4.2.1 GRAFOS NÃO PONDERADOS 31
4.2.2 GRAFOS PONDERADOS 33
4.3 EXPERIMENTAÇÃO E ANÁLISE DOS RESULTADOS 36

5 RESULTADOS E DISCUSSÃO 38

6 CONCLUSÃO 44
6.1 PERSPECTIVAS FUTURAS 44

REFERÊNCIAS 46

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

1 INTRODUÇÃO

Os avanços tecnológicos envolvendo visão computacional, eletrônica e inteligência artificial, entre outras áreas correlatas, impulsionaram a tendência de desenvolvimento e uso de ferramentas que automatizam as disputas em ambientes de combate. Nesse sentido, com a possibilidade de aumentar a eficácia operacional e reduzir a participação e desgaste humano — física, cognitiva e psicologicamente — no campo de batalha, a integração de robôs autônomos nesses cenários de confronto é iminente e torna-se motivo de vantagem tática e estratégica.

Em relação à autonomia desses sistemas, o planejamento de trajetória de robôs terrestres é uma das tarefas essenciais a ser realizada e ela se encontra dentro de um escopo multidisciplinar que inclui conhecimentos em diversas áreas de grande complexidade, como a robótica e a teoria dos grafos. Essas disciplinas convergem na modelagem do cenário de combate como um grafo, no qual os nós representam as posições transitáveis pelos robôs no mapa e as arestas definem os caminhos. Dessa maneira, algoritmos de busca e otimização são ferramentas que guiam o planejamento das possíveis rotas, auxiliando na coordenação entre pontos e adaptação aos obstáculos, otimizando o funcionamento e uso no campo de combate.

A pesquisa “Cooperação Entre Plataformas Terrestres e Aéreas para Aquisição, Rastreamento e Perseguição de Alvos, Utilizando Visão Computacional e Inteligência Artificial” (1) que se encontra em fase de desenvolvimento no laboratório LIARC, iniciada em 2023 pelo Major Fábio Luiz Junior, aborda o emprego de plataformas robóticas aéreas (UAV — Veículos Aéreos Não Tripulados) e terrestres (UGV — Veículos Terrestres Não Tripulados) em ambientes de batalha e como essas duas estruturas podem trabalhar em conjunto para otimizar e ampliar o poder de ataque e estratégico militar: UAV sendo responsável pelo mapeamento geral do terreno e identificação rápida de obstáculos e forças inimigas; e o UGV, a partir das informações obtidas previamente, com a função de realizar investigações mais detalhadas a respeito do local de combate e de executar ações desejadas conforme necessário. Nessa pesquisa, o autor aborda o desafio que o planejamento de trajetória representa para o campo da robótica e a sua importância para a navegação de maneira autônoma e segura em um ambiente (1).

Nesse sentido, o presente projeto, intitulado “Planejamento de trajetória para robô terrestre, utilizando otimização em grafos”, busca, em um contexto maior e em linha com a pesquisa em andamento, aprofundar e colaborar com a tarefa e sua aplicação no contexto de otimizar caminhos de planejamento de trajetória de robôs.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

1.1 Motivação e Justificativa

O presente projeto busca colaborar e aprofundar o estudo de planejamento de rotas abordado em (1) e, que por sua vez, poderá ser aproveitado e aplicado no projeto estratégico "Sistemas de Veículos Terrestres Remotamente Pilotados" (SVTRP), conduzido pelo Centro Tecnológico do Exército (CTEx), visando a uma potencialização de seu funcionamento.

O aprimoramento da autonomia de plataformas robóticas terrestres (UGVs) representa uma evolução tecnológica relevante para o Exército Brasileiro (EB) e é motivo de fortalecimento de vantagem estratégica e tática. Diante disso, o desenvolvimento de algoritmos para melhorar o processo de encontrar trajetórias otimizadas nesse contexto contribui para aprimorar a tecnologia e fortalecer o poder de combate das plataformas robóticas terrestres.

1.2 Objetivos

O principal objetivo deste Projeto de Final de Final Curso consiste em auxiliar o aprimoramento do sistema proposto em (1) com a inclusão de diversos algoritmos de planejamento de trajetória. Esses algoritmos visam viabilizar a geração de caminhos entre a posição atual do robô terrestre e um alvo predeterminado.

Para atingir o objetivo principal, foram definidos os seguintes objetivos específicos:

- Realizar a análise dos diferentes tipos de algoritmos disponíveis na literatura e aplicados ao planejamento de trajetórias;
- Selecionar os algoritmos mais adequados ao problema descrito em (1); e
- Executar testes comparativos e avaliar o desempenho dos algoritmos selecionados por meio de conjuntos de dados apropriados.

1.3 Metodologia

Visando alcançar os objetivos propostos, esta seção descreve detalhadamente a metodologia adotada ao longo deste projeto.

Inicialmente, foi conduzida uma revisão sistemática da literatura, com foco no trabalho de Fábio Luiz (1) sobre planejamento de trajetória utilizando o Rapidly-exploring Random Tree (RRT). Adicionalmente, foram estudados algoritmos clássicos de grafos aplicados ao planejamento de trajetórias em robótica — em particular, Breadth-First Search (BFS), Dijkstra e A*.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Na etapa seguinte, procedeu-se ao levantamento de requisitos para delimitar as funcionalidades essenciais que o sistema deveria apresentar.

Foi realizada a implementação em Python dos algoritmos Dijkstra e A* para grafos não ponderados. Utilizou-se como referência o formato de entrada e saída proposto em (1), e definiram-se métricas objetivas de desempenho para avaliar comparativamente cada método.

Concluída a fase inicial de codificação, os algoritmos foram aplicados a mapas segmentados obtidos por sensoriamento e disponibilizados para teste; e avaliou-se, separadamente, cada resultado alcançado.

Na etapa seguinte, definiram-se os critérios de ponderação a serem aplicados às arestas do grafo e os pesos foram incorporados na implementação. A partir dessa modificação, realizaram-se novas execuções utilizando algoritmos adequados a grafos ponderados. Os resultados obtidos foram analisados comparativamente tanto entre os diferentes algoritmos quanto em relação aos resultados previamente gerados na versão não ponderada do grafo.

1.4 Estrutura

Este documento está organizado da seguinte forma:

No Capítulo 2, são apresentados os conceitos preliminares que servirão de base para o desenvolvimento do projeto.

O Capítulo 3 traz uma revisão da literatura, com foco na análise do estado da arte em planejamento de trajetórias para robôs terrestres, além de uma avaliação do estado atual do projeto SVTRP e a definição do posicionamento em relação a ele.

No Capítulo 4, descrevem-se os procedimentos adotados para a coleta de dados, implementação dos algoritmos e análise dos resultados obtidos.

O Capítulo 5 é dedicado à apresentação e discussão dos resultados, com a comparação do desempenho dos algoritmos propostos e a análise de sua aplicação em um ambiente de teste simulado.

Por fim, no Capítulo 6, são apresentadas as conclusões do trabalho, bem como sugestões de possíveis extensões futuras para o projeto.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

2 CONCEITOS PRELIMINARES

Neste capítulo, exploraremos alguns conceitos importantes para o entendimento do projeto.

2.1 Veículos Terrestres Autônomos

Segundo (2), um veículo terrestre autônomo (UGV) é um sistema robótico complexo e multidisciplinar, com participação ostensiva das engenharias mecânica, da computação e eletrônica. Este sistema têm como objetivo navegar pelo terreno de forma autônoma, sem a necessidade de intervenção por um operador externo, e executar determinada tarefa. Nos últimos anos, evidencia-se grande crescimento nas aplicações de UGVs, desde aplicações civis, como robôs domésticos e veículos autônomos, até aplicações militares, como robôs terrestres para combate.

A seguir, descrevemos alguns componentes chave de veículos autônomos:

Sensores: Recebem informações importantes para controle do sistema. Podem ser sensores internos (odômetro, sensor de pressão), externos (bússola, GPS) e ambientais (usados para captar a posição do veículo relativa ao ambiente).

Sistemas de comunicação: Entre um conjunto de veículos ou entre o veículo e um controlador.

Sistemas de navegação: Permitem a localização do veículo no espaço, cálculo de rotas, transcrição em coordenadas geográficas. Sistema essencial que promove a autonomia do veículo e automatiza os deslocamentos do mesmo.

2.2 Planejamento de Caminhos

O planejamento de caminhos é considerado um desafio crítico em Inteligência Artificial (IA) e essencial para o movimento autônomo de todos os sistemas inteligentes, como veículos terrestres. Algoritmos para planejamento de caminho buscam estabelecer um conjunto de caminhos e movimentos para que um determinado veículo móvel atinja um objetivo, navegando autonomamente pelo ambiente.

Os termos planejamento de trajetória e planejamento de caminho muitas vezes podem ser utilizados intercambiavelmente, porém possuem algumas nuances em sua semântica. Este, mais comumente, referencia o processo de encontrar uma curva no espaço, com pontos inicial e final bem definidos. Esta curva será formada por um conjunto de segmentos – o caminho retornado pelos algoritmos. Entretanto, somente esta curva não

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

necessariamente é suficiente para garantir a habilidade do sistema de atingir o objetivo, devido às características físicas dos sistemas robóticos. O termo planejamento de trajetória está intrinsecamente ligado à consideração das características dinâmicas do sistema. Não determina apenas aonde o veículo move, mas também considera o como ele percorrerá este caminho (3).

O planejamento de trajetórias pode ser categorizado de duas maneiras. Ele é considerados global ou estático, caso toda a informação sobre o ambiente seja conhecida, e local ou dinâmico, quando o ambiente é totalmente ou parcialmente desconhecido, com possíveis obstáculos que não seriam considerados no planejamento estático (4). Neste trabalho, estaremos interessados no planejamento de caminhos globais.

Para compararmos os caminhos encontrados, normalmente são usados os termos viabilidade – encontrar um caminho seguro entre os pontos – e otimalidade – encontrar um caminho com menor custo ou distância. Outro termo relevante é a completude dos métodos. Dizemos que um algoritmo é completo quando é capaz de encontrar uma solução, se ela existe ou indicar a falta dela, caso nenhum caminho seja viável.

2.3 Grafos

Um grafo $G = (V, E)$ consiste em um conjunto de vértices, também denominados nós, $V = \{v_1, v_2, v_3, ...\}$ e um conjunto de arestas $E = \{e_1, e_2, e_3, ...\}$, tal que cada aresta é identificada com um par não ordenado de vértices (v_i, v_j). Dizemos que v_i e v_j são vizinhos caso a aresta (v_i, v_j) exista. Os vértices associados a uma aresta e_k são denominados extremidades de e_k (5). Para diversas aplicações, é interessante associarmos um valor a cada aresta, chamado de peso. Podemos então classificar os grafos como ponderados, caso suas arestas possuam pesos associados, ou não ponderados, caso contrário.

Um passeio em grafo é definido como uma sequencia finita e alternada de vértices e arestas, de tal forma que cada aresta tenha como extremidades os vértices predecessor e sucessor na sequência. Um caminho é um passeio aberto (vértice inicial é diferente do final) onde não há repetição de vértices. Dizemos que um grafo é conexo quando existe pelo menos um caminho entre qualquer par de vértices. Podemos então definir a distância entre dois vértices como o número (ou soma dos pesos associados as arestas) em um caminho mínimo que conecta eles.

2.4 Algoritmos de Caminhos Mínimos em Grafos

O problema de caminhos mínimos é o problema de encontrar a menor rota entre um ponto inicial e um ponto final. Um grande número de problemas de otimização é equivalente a encontrar caminhos mínimos em grafos, por isso, algoritmos de minimização

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

de caminho são extensivamente explorados na literatura. Existem diferentes tipos de problemas de caminhos mínimos, mais frequentemente encontrados entre as 5 categorias a seguir(6):

1. Caminhos mínimos entre dois vértices específicos
2. Caminhos mínimos entre todos os pares de vértices
3. Caminhos mínimos entre um vértice especifico e todos os outros
4. Caminhos mínimos entre dois vértices específicos, que passam por um determinado conjunto de vértices
5. Caminhos mínimos secundários, terciários, e assim por diante.

Neste projeto, vamos abordar na primeira classe de problemas. Abaixo exploramos alguns destes algoritmos.

2.4.1 Breadth-First Search (BFS)

A busca em largura (Breadth-First Search), é a fundação de vários algoritmos em grafos, e é comumente utilizado para testar conectividade em um grafo ou encontrar os caminhos mínimos a partir de um único vértice em um grafo sem pesos. Iniciando no nó fonte, a fronteira de busca expande em cada passo, visitando todos os nós em uma mesma profundidade (número de arestas no caminho entre a fonte e o vértice), antes de visitar qualquer nó mais profundo. O passo iterativo verifica, para cada nó seus nós vizinhos, os quais são adicionados à fronteira de busca, o nó atual é marcado visitado e definido como pai dos vizinhos. Este algoritmo gera uma árvore para o componente conexo do grafo que contém o vértice fonte. (7)

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Algorithm 1 $BFS(G = (V, E), s, t)$

```
1: visited ← False para todo v ∈ V
2: parent ← Null para todo v ∈ V
3: queue ← ∅
4: visited(s) ← True
5: queue ← queue ∪ {s}
6: while queue ≠ ∅ do
7:    v ← primeiro elemento de queue
8:    remova v de queue
9:    if v = t then
10:      break
11:   for u ∈ N_G(v) do
12:      if visited(u) = False then
13:         visited(u) ← True
14:         parent(u) ← v
15:         queue ← queue ∪ {u}
16: return parent
```

Algorithm 2 $RECUPERA_CAMINHO(parent, s, t)$

```
1: path ← [ ]
2: current ← t
3: while current ≠ Null do
4:    insira current no início de path
5:    if current = s then
6:       break
7:    current ← parent(current)
8: if path[1] ≠ s then
9:    return [] {não existe caminho}
10: return path
```

No Algoritmo 1, implementamos algumas mudanças considerando nossa aplicação, como retornar um dicionário com os nós pais para cada vértice do grafo – com isso é possível reconstruir o caminho – e interromper adicionar um critério de parada quando se atinge o nó destino, truncando o tempo de execução. Foi utilizada uma fila para armazenar os nós na fronteira de busca. De qualquer maneira, o BFS ainda apresenta suas limitações, pois explora muitos caminhos desnecessáriamente.

O Algoritmo 2 recebe um dicionário relacionando cada nó com seu pai e constrói o caminho entre a fonte s e o destino t, caso ele exista.

2.4.2 Algoritmo de Dijkstra

O algoritmo de Dijkstra para o caminho mínimo entre dois vértices se baseia na seguinte intuição: Se R é um nó no caminho mínimo entre P e Q, conhecimento deste

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

implica no conhecimento do caminho mínimo entre P e R. Então, para encontrar o caminho mínimo, basta construir os caminhos mínimos a partir de P na ordem crescente de distância, até que seja atingido o vértice Q (8). Esta estratégia funciona quando o grafo não possui arestas com peso negativo.

A busca pelo caminho mínimo se dá de forma similar ao BFS, de forma que, começando no nó fonte, adicionamos seus vizinhos na fronteira de busca e, a cada passo, exploramos um nó da fronteira. A grande diferença é a inclusão de um critério de escolha do próximo vértice. No algoritmo de Dijkstra, seleciona-se aquele com menor distância percorrida até o momento para expandir a fronteira de busca.

Algorithm 3 $DIJKSTRA(G = (V, E), w, s, t)$

```
1: dist ← ∞ para todo v ∈ V
2: parent ← Null para todo v ∈ V
3: dist(s) ← 0
4: Q ← V {conjunto de vértices não visitados}
5: while Q ≠ ∅ do
6:    u ← vértice em Q com menor dist(u)
7:    remova u de Q
8:    if u = t then
9:       break
10:   for v ∈ N_G(u) tal que v ∈ Q do
11:      alt ← dist(u) + w(u, v)
12:      if alt < dist(v) then
13:         dist(v) ← alt
14:         parent(v) ← u
15: return (dist, parent)
```

O algoritmo retorna a distancia entre a fonte e cada os nós explorados e uma lista com os pais na árvore correspondentes a esses nós. Novamente, truncamos a execução ao encontrar o nó destino. O algoritmo de Dijkstra, no pseudocódigo apresentado como Algoritmo 3, assim como o BFS, explora o grafo construindo uma árvore de caminhos entre um nó e todos os outros, por isso adicionamos esta medida como forma de acelerar o término da execução. Podemos utilizar o Algoritmo 2 para recuperar o caminho entre a fonte e o destino.

2.4.3 Algoritmo A*

Em (9), Hart et al propõem um algoritmo baseado em heurística para resolver o problema de caminhos mínimos em um grafo. Uma abordagem heurística tipicamente usa conhecimento especial sobre o domínio do problema representado no grafo para tentar melhorar a eficiência da solução. Entretanto, procedimentos baseados em heurística podem não garantir que uma solução ótima seja sempre encontrada. O algoritmo A* é um

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

algoritmo geral, que concilia as abordagens heurística e matemática (sem conhecimento de fatores externos ao grafo). (9)

O A* também visita um vértice e adiciona seus vizinhos na fronteira de busca. A diferença se dá no critério para escolha do próximo vértice n a ser visitado. Neste caso, usa-se o menor valor da função de avaliação: $f(n) = g(n) + h(n)$, onde s é o nó de início, $g(n)$ é o custo do caminho da fonte s ao vértice n encontrado até o momento e $h(n)$ é a estimativa do custo de um caminho ótimo até o destino t.

Para uma heurística admissível, o A* garante otimalidade. Ou seja, se a heurística $h(n)$ nunca subestimar o custo do menor caminho entre n e o destino, o A* sempre encontra uma solução de custo mínimo (ou garante a inexistência dela), examinando o menor número necessário de nós.

Algorithm 4 $A^*(G = (V, E), w, h, s, t)$

```
1: g_score ← ∞ para todo v ∈ V {custo do caminho de s até v}
2: f_score ← ∞ para todo v ∈ V {estimativa total: g + h}
3: parent ← Null para todo v ∈ V
4: g_score(s) ← 0
5: f_score(s) ← h(s)
6: openSet ← {s} {nós a explorar}
7: while openSet ≠ ∅ do
8:    current ← nó em openSet com menor f_score(current)
9:    if current = t then
10:      break
11:   remova current de openSet
12:   for neighbor ∈ N_G(current) do
13:      tentative_g ← g_score(current) + w(current, neighbor)
14:      if tentative_g < g_score(neighbor) then
15:         parent(neighbor) ← current
16:         g_score(neighbor) ← tentative_g
17:         f_score(neighbor) ← g_score(neighbor) + h(neighbor)
18:         if neighbor ∉ openSet then
19:            openSet ← openSet ∪ {neighbor}
20: return (g_score, parent)
```

A grande vantagem do A* em comparação aos algoritmos supracitados é a consideração conjunta do caminho percorrido até o momento e a previsão do caminho restante, dada pela função de avaliação. Ela garante explorar o menor número de nós possível, quando a heurística usada for admissível. No problema de planejamento de trajetória para robôs, existem duas heurísticas comumente usadas, a depender da implementação (considerando coordenadas bidimensionais): a distância euclidiana ($\sqrt{x^2 + y^2}$) ou a distância de Manhattan ($|x| + |y|$). O Algoritmo 4 apresenta o pseudocódigo de A*. O Algoritmo 2 novamente deve ser utilizado para recuperar o caminho.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

2.5 Rapidly-exploring Random Tree (RRT)

O algoritmo Rapidly-exploring Random Trees (RRT) é um método de planejamento de trajetórias que combina características de busca e de amostragem. A características de busca está presente no fato de que o algoritmo inicia em um nó e constrói ramificações, como uma árvore, até atingir o nó destino. Durante a ramificação, porém, os pontos são escolhidos aleatoriamente, característica de algoritmos baseados em amostragem. Desta forma, o RRT consegue alcançar uma performance melhor que outros algoritmos e não encontra problemas relacionados a máximos locais. Entretanto, a natureza estocástica do algoritmo resulta em grandes diferenças entre resultados e não garante encontrar um caminho ótimo (10). Para contornar este problema, normalmente se executa o RRT multiplas vezes para o mesmo conjunto de dados de entrada e seleciona-se o melhor resultado entre eles.

Algorithm 5 $RRT(s, t, passo, max_iters)$

$V \leftarrow \{s\}$ {conjunto de nós}
$parent \leftarrow$ vetor com $parent(s) \leftarrow Null$
for $i = 1$ to max_iters **do**
  $x_{rand} \leftarrow$ ponto aleatório no espaço de busca
  $x_{nearest} \leftarrow$ nó em V mais próximo de x_{rand}
  $x_{new} \leftarrow$ ponto em direção a x_{rand} a partir de $x_{nearest}$ com passo
  if x_{new} é válido (sem colisão) **then**
    adicione x_{new} em V
    $parent(x_{new}) \leftarrow x_{nearest}$
    if x_{new} está suficientemente perto de t **then**
      break
return $parent$

O algoritmo RRT escolhe um ponto aleatório no espaço de busca e, a partir do nó explorado mais próximo deste, se desloca uma distância fixa nessa direção, caso não haja nenhuma colisão neste caminho. Desta forma, uma árvore vai sendo criada com a adição de novos pontos em uma direção aleatória. O RRT não garante que encontrará uma solução, e não consegue identificar a inexistência da mesma, por isso, um dos parâmetros que o algoritmo recebe é o número máximo de iterações, para interromper sua execução.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

3 REVISÃO DE LITERATURA

No início da elaboração deste projeto, foi realizada a pesquisa bibliográfica no que concerne aos conceitos abordados.

3.1 Revisão Bibliométrica

A revisão bibliométrica foi focada em artigos científicos publicados nos últimos cinco anos, entre janeiro de 2021 e maio de 2025. A pesquisa foi orientada para publicações que envolvessem planejamento de trajetórias, grafos, algoritmos e veículos terrestres não tripulados — do inglês, Unmanned Ground Vehicle (UGV). A seguir está representada a expressão de busca utilizada.

```
[All: ugv] AND [All: path planning] AND [All: graph] AND [All: algorithm]
        AND [E-Publication Date: (01/01/2021 TO 01/05/2025)]
```

Os resultados obtidos com essa busca foram 71 trabalhos na base da **ACM**[1] e 28 na base da **IEEE**[2]. A partir da consolidação de todos esses resultados, foi criado o histograma da Figura 2 que apresenta a quantidade desses trabalhos por ano de publicação.

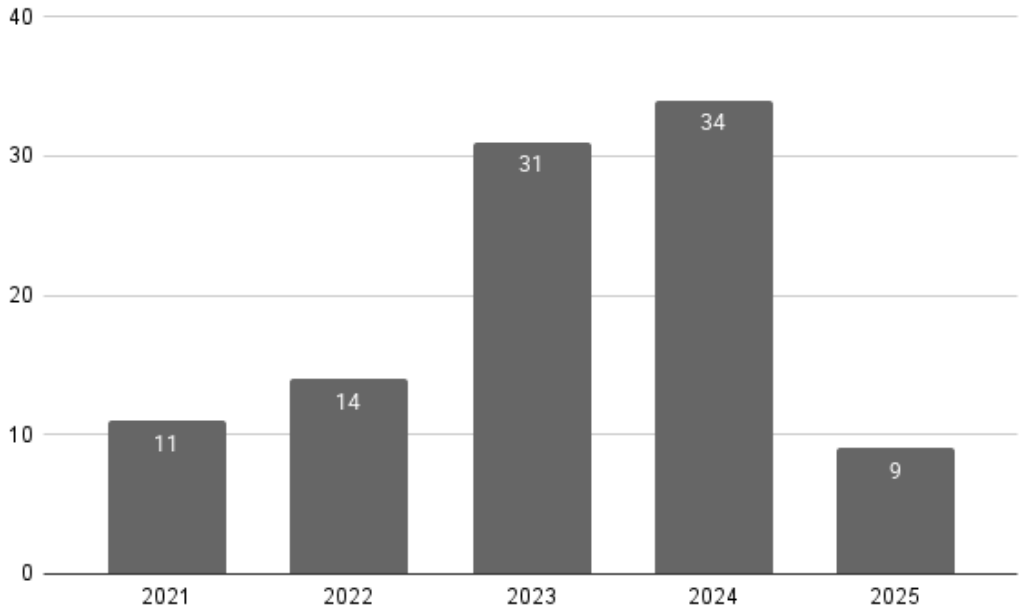


Figura 1 – Histograma de distribuição de trabalhos por ano

[1] https://dl.acm.org
[2] https://ieeexplore.ieee.org

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

A partir da análise da Figura 1, nota-se uma tendência de aumento na quantidade de publicações entre 2021 e 2024 — a lista de publicações do ano de 2025 ainda não está completa. Nesse sentido, podemos concluir que o tema deste projeto, ultimamente, tem ganhado considerável atenção e está sob um crescente interesse e intensificação do volume de pesquisas.

A Figura 2 destaca os países mais envolvidos na elaboração desses trabalhos, mostrando a produção de cada um.

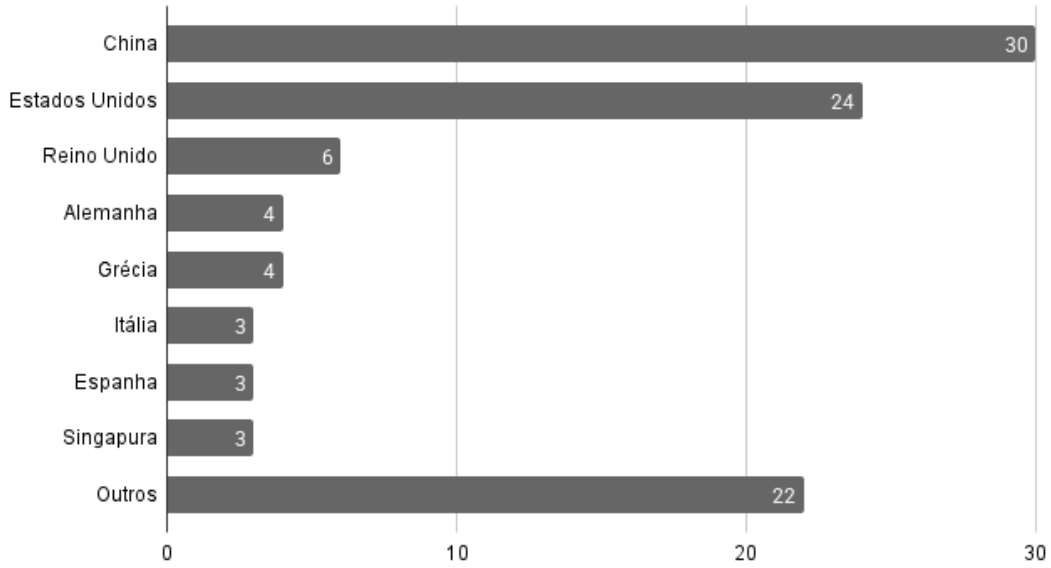


Figura 2 – Distribuição de trabalhos por país

A Figura 2 evidencia que a grande maioria dos estudos se concentra na China e nos Estados Unidos, enquanto outros países apresentam participação significativamente menor; dentre estes, o Reino Unido se destaca, embora contabilize apenas seis publicações.

Além disso, foi realizada uma análise dos principais tópicos abordados nos trabalhos identificados na busca, por meio do mapeamento de suas palavras chave. Elas foram consolidadas e ajustadas para unificar possíveis sinônimos — substituindo ‘path-planning‘ ou ‘trajectory planning‘ por ‘Path Planning‘, por exemplo — e, posteriormente, contabilizadas para estudo da frequência de aparições, conforme ilustrado no gráfico da Figura 3.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

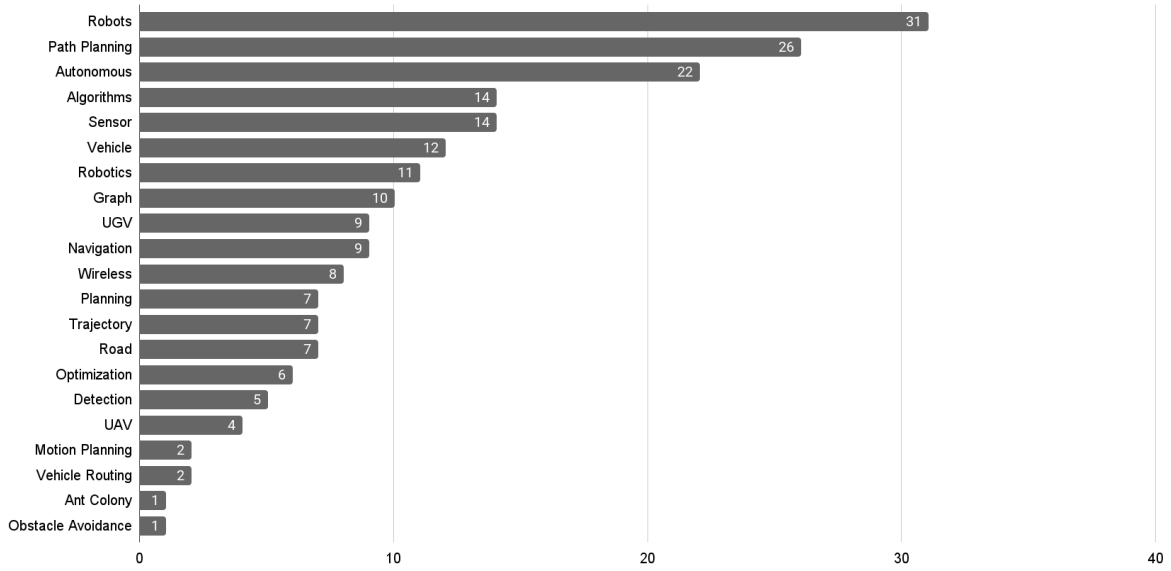


Figura 3 – Frequência de aparições de palavras chave

Nota-se que robótica, planejamento de trajetórias, autonomia, algoritmos e grafos aparecem com frequência como palavras chave nos trabalhos analisados.

3.2 Revisão Bibliográfica

Na literatura, a integração entre veículos aéreos não tripulados (UAV) e veículos terrestres não tripulados (UGV) é uma ideia explorada recorrentemente, principalmente a partir da última década. Como (11) reforça, o uso de drones aéreos já foi bastante disseminado, com aplicações diversas nas áreas civis e militares. Entretanto, existem situações em que drones aéreos não são suficientes para executar determinada tarefa. Nesse contexto, explora-se uma abordagem híbrida, empregando também veículos autônomos terrestres, capazes de acessar áreas inalcançáveis de outra forma. Para isto, UAVs são normalmente utilizados para fazer a fotogrametria do terreno, e os mapas gerados alimentam um sistema de planejamento de caminhos, como explicita Li et al. em (12).

Em seu paper, Li et al. estudou a captura de imagens e o consequente planejamento de caminhos para um sistema híbrido. As imagens capturadas com o auxílio de um UAV equipado com câmeras e sensores precisaram passar por um preprocessamento, de forma a atingir uma qualidade aceitável para alimentar os algoritmos de planejamento de caminhos. Foram utilizados métodos para remoção de ruídos, correção da imagem e identificação de obstáculos.

No contexto mais geral de colaboração UAV-UGV, surgem alguns problemas quando começamos a expandir as aplicações. Em (11), o planejamento de um caminho circular

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

aumentou consideravelmente a complexidade do problema, sendo necessária a modelagem do mesmo como um problema de otimização em larga escala. Dependendo do tipo de aplicação, limitações de alcance podem se tornar um problema, como é mostrado em (13), com a dificuldade de manter o UAV carregado durante todo o percurso.

Em relação ao planejamento de caminhos para UGVs, há inúmeras revisões dos diferentes algoritmos existentes. Dito isto, o desenvolvimento de novos algoritmos ainda é extremamente presente na literatura, dada a importância da criação de algoritmos individualizados para cada classe de problemas. Este fato explica a grande heterogeneidade dos algoritmos propostos na literatura, desde algoritmos exatos, como o A*, baseados em amostragem, como o RRT, até algoritmos baseados na biologia, como algoritmos genéticos.

Uma abordagem muito utilizada é a aplicação de um algoritmo global, que atuará em um ambiente estático e resultará em um resultado preliminar, seguido de otimizações locais, que visam lidar com características mais dinâmicas do ambiente. (14) explora esta questão utilizando um algoritmo baseado em biologia para criação do caminho, utilizando um método baseado em amostragem para adaptá-lo na presença de obstáculos não previstos, desviando do caminho original, mas evitando colisões. (15) utiliza esta ideia para tratar do problema de colisão em sistemas multi-robô. Utiliza-se um algoritmo novamente baseado em biologia para criação dos caminhos, com uma estratégia probabilística para otimizar o caminhos de cada UGV. Em seguida, um algoritmo de otimização de mudança de velocidade é aplicado para impedir colisões.

Outro caso específico pode ser visto em (16). Neste artigo, é discutido a viabilidade do uso de algoritmos clássicos para planejamento de caminhos em uma situação de dificuldade de travessia. É proposto um algoritmo A* híbrido, que busca caminhos ótimos aproximados baseados em um mapa de custo não uniforme baseado na dificuldade de travessia de determinado terreno.

Em (3), foi feita uma extensiva revisão sobre os principais algoritmos que servem de base para as especializações apresentadas. Também estão comparados os algoritmos propostos nesse projeto. Uma sumarização deste artigo pode ser dado pela tabela retirada do mesmo:

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Method	**Algorithm**	**Advantages**	**Disadvantages**
Decomposition graph-based	Dijkstra	• Easy implementation • Optimal solution in discretized space • Multiple environments	• High computational cost • Static environments
	A*	• Optimal solution in discretized space • Guided and fast search • Online implementation	• High computational cost in large environments • Static environments
	D*	• Optimal solution in discretized space • Guided and fast search • Dynamic environments	• Heuristic function uses unrealistic distances
Sampling-based	PRM	• Reusable solution • Fast exploration of large environments	• Non-optimal solution • Passive method
	Voronoi	• Easy implementation • Low computational cost	• Passive method • Nonconvergence • Static environments
	RRT	• Low computational cost • Fast exploration	• Single path generation
	APF	• Low time complexity • Online implementation	• Local minima
Mathematic model-based	MILP	• Complete system and environmental information	• High time complexity
	MIQP	• Time consideration for path planning	• Complex mathematical model solving
	OC	• Solution considers the control of the systems	• No analytical solutions
Bioinspired	NN	• Infinite environments and systems	• Non-generic solutions • Success conditional on correctly parameterized rules and organisms
	EA	• Suitable for NP-hard and multi-objective problems	• High time complexity

Tabela 1 – Comparação de algoritmos de planejamento de trajetória baseados em diferentes abordagens.

Entretanto, decidir entre cada algoritmo não é uma tarefa trivial. Para cada aplicação, deve-se analisar a viabilidade de determinado algoritmo em relação à otimalidade,

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

tempo máximo de execução e à capacidade computacional do sistema. Utilizando mecanismos de comunicação entre partes do sistema, por exemplo, podemos extrair o planejamento de caminhos das unidades em campo, centralizando em um controlador central e apenas distribuir o resultado dele. Assim, podemos concentrar os passos mais custosos em uma máquina mais potente, podendo atingir um melhor valor de objetivo, sem sacrificarmos tempo de execução para isso. Neste projeto, consideramos o cálculo dos caminhos feitos de forma centralizados.

Na pesquisa "Cooperação Entre Plataformas Terrestres e Aéreas para Aquisição, Rastreamento e Perseguição de Alvos, Utilizando Visão Computacional e Inteligência Artificial" (1), é proposta uma estação de controle em solo para centralizar o controle dos sistemas robóticos distribuídos no terreno. Assim, um algoritmo de trajetória mais robusto pode ser executado centralizadamente.

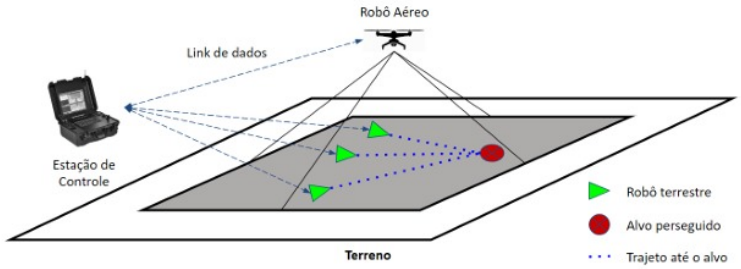


Figura 4 – A imagem retirada de (1) ilustra um robô aéreo sobrevoando o terreno da missão. Os triângulos verdes representam um conjunto de robôs terrestres e o círculo vermelho representa o alvo da missão. A operação é controlada por um operador na estação de controle em solo.

Nela, a metodologia para o planejamento de trajetórias se divide em três passos. Inicialmente é modelado o ambiente a ser percorrido – isto é feito com auxílio de drones e inteligência artificial. Então executa-se um algoritmo de planejamento de caminhos – RRT, neste caso – e, finalmente, adequa-se o caminho encontrado para satisfazer as restrições cinemáticas do robo. O presente projeto terá como principal área de atuação a etapa 2 desta metodologia – planejamento automático de caminhos.

Para a modelagem do terreno, é feito um voo de drone que captura imagens de uma visão superior do ambiente. Essas imagens são então tratadas por inteligencias artificiais (IA) treinadas em um grande dataset publico de imagens exemplo. O aprendizado de máquina aplicado nesses dados ensina, a essas IAs, a segmentação semântica do terreno, diferenciando entre áreas transitáveis e não transitáveis. Uma descrição mais detalhada deste processo pode ser encontrada em (1).

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

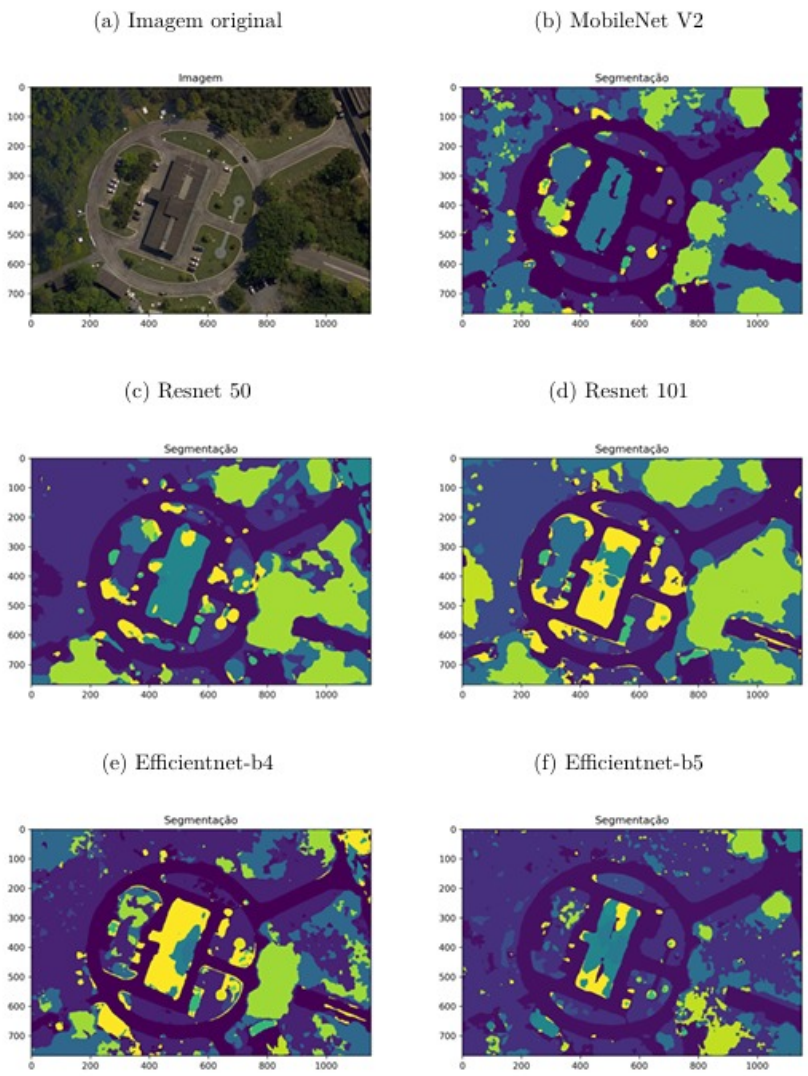


Figura 5 – As imagens segmentadas obtidas utilizando as cinco arquiteturas de redes diferentes em (1).

As imagens processadas alimentam um sistema de planejamento de trajétórias, que no caso supracitado utiliza o algoritmo Rapidly-exploring Random Trees (RRT) para gerar os caminhos. Este cálculo de rotas é feito repetidas vezes para contornar uma limitação do RRT – sua escolha aleatória para determinar os pontos do caminho. Em seguida, é feita uma suavização da trajetória para se adequar às limitações físicas do robô.

A literatura analisada carece de análises quantitativas de diferentes algoritmos, com revisões quantitativas, como em (3) e (10), recebendo maior enfoque. Nesse contexto, o presente estudo busca estimular esta linha de pesquisa, pela comparação quantitativa do uso de algoritmos baseados em grafo para planejamento de caminhos para robôs terrestres.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

4 METODOLOGIA

No presente capítulo, abordaremos os métodos utilizados para coleta de dados de diferentes imagens aéreas segmentadas semanticamente; desenvolvimento dos algoritmos A* e Dijkstra; execução desses algoritmos e do RRT sobre as imagens obtidas; e análise e comparação dos resultados obtidos. Também serão apresentadas as configurações da máquina utilizada para obtenção dos caminhos.

4.1 Coleta dos Dados (Mapeamento)

Para comparar os diferentes algoritmos de planejamento de trajetória, foram usadas imagens aéreas produzidas em (1) e imagens disponibilizadas em (17). A entrada dos algoritmos escolhidos são imagens reais, processadas e semanticamente segmentadas.

A segmentação semântica permite a separação das áreas de uma imagem aérea em diferentes classes, dependendo do tipo de terreno que cada parte representa. Cada classe é associada a uma cor única para identificação e os respectivos pixels são recoloridos para refletir essa classificação. Assim, podemos discernir entre áreas transitáveis (onde o UGV é capaz de se locomover livremente, como campos abertos, estradas e superfícies uniformes no geral) e não transitáveis (apresentam algum tipo de obstáculo para o trânsito do veículo, como árvores, edifícios ou outros obstáculos no geral).

4.2 Construção dos grafos

O objetivo da etapa de construção dos grafos é organizar as informações retiradas da imagem segmentada em uma estrutura de dados adequada aos algoritmos. Tanto a criação e manipulação dos grafos quanto os algoritmos foram inteiramente implementados na linguagem de programação Python pelos autores. Os experimentos foram realizados em um Apple Macbook Pro com 24 gigabytes de memória e processador M4 Pro.

4.2.1 Grafos não ponderados

Com o uso da ferramenta de visão computacional *OpenCV*, processamos por inteiro as imagens segmentadas. Esse processamento consiste na obtenção da cor de cada pixel separadamente e na comparação dessas cores com a cor correspondente às áreas transitáveis, com base no modelo de cores BGR (Blue, Green, Red), onde caracterizamos um pixel como transitável se o seu código BGR for equivalente ao código correspondente às áreas transitáveis definidas na segmentação. Dessa maneira, todos os pixels transitáveis da imagem são considerados nós do grafo, enquanto os restantes são definidos como obstáculos.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

A partir da criação dos nós, as arestas são definidas automaticamente entre pixels adjacentes, com base no input fornecido em tempo de execução para a escolha do tipo de movimento: 4 sentidos (horizontais e verticais); ou 8 sentidos (horizontais, verticais e diagonais). Por padrão, as análises realizadas neste projeto consideram os 8 sentidos.

Os algoritmos 6, 7 e 8 representam os pseudocódigos dos elementos participantes na criação do grafo, onde o Algoritmo 6 é a função diretamente responsável por essa construção. Ele obtém as dimensões altura (h) e largura (l) da imagem do mapa segmentado, que equivale a uma matriz $h \times l$ de quadrados de lados unitários; e, para cada um desses quadrados, verifica se sua cor é equivalente à cor correspondente às áreas transitáveis, por meio do Algoritmo 7. Caso as cores sejam iguais, o quadrado (x,y) da matriz corresponde a um nó do grafo, representado pelo Algoritmo 8; caso contrário, é um objeto nulo.

Algorithm 6 $CREATE_TREE(img, transitable_color)$

```
(h, l, _) ← dimensões de img
node_list ← lista vazia
for y ← 0 to h − 1 do
    linha ← lista vazia
    for x ← 0 to l − 1 do
        if IS_TRANSITABLE(x, y, img, transitable_color) then
            adicionar novo Node() em linha
        else
            adicionar NULL em linha
    adicionar linha em node_list
return node_list
```

Algorithm 7 $IS_TRANSITABLE(x, y, img, transitable_color)$

```
cor ← img[y][x]
if cor = transitable_color then
    return TRUE
else
    return FALSE
```

Algorithm 8 Classe $Node$

```
Classe Node:
    parent_x ← −1
    parent_y ← −1
Construtor: inicializa os atributos acima
```

Os veículos terrestres não podem ser considerados adimensionais, pois eles possuem dimensões físicas reais (largura, comprimento, diâmetro de giro), que influenciam diretamente sua capacidade de locomoção. Portanto, a simples classificação de um nó como transitável não garante a viabilidade da inclusão do mesmo em um caminho a ser

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

percorrido pelo robô, pois há a possibilidade de ser um caminho muito estreito, ou com algum obstáculo próximo e não comportar as dimensões do veículo.

A representação em grafo não ponderado ignora este fato, sendo propensa a gerar caminhos não desejados, que podem impactar no mau desempenho quando aplicada a situações reais de planejamento de trajetória. Para isso, uma nova abordagem é proposta, onde passamos a ter um grafo ponderado que reflete as limitações discutidas.

4.2.2 Grafos ponderados

A definição do grafo ponderado leva em consideração a localização de cada região transitável em relação aos obstáculos próximos, uma vez que a proximidade a estes implica em um maior risco na travessia daquelas. Para isso, inicialmente, é aplicada uma máscara binária sobre o mapa da imagem, onde áreas transitáveis recebem o valor 1 (branco) e áreas não transitáveis recebem 0 (preto).

Durante a criação do grafo dos nós transitáveis, é feita uma verificação iterativa dos pixels ao redor do nó que verifica a distância dele ao obstáculo mais próximo, com base em um parâmetro definido em tempo de execução que representa a dimensão do robô. Durante a análise, os nós que estão com uma distância a algum obstáculo menor do que este parâmetro são descartados do grafo (o veículo nunca conseguirá ocupar aquela posição), recebendo um valor de peso que simula o infinito. Em relação ao restante dos pontos, é associado um valor de peso a cada nó que reflete a dificuldade de transitar por ele dada a proximidade a um obstáculo, seguindo a fórmula: $max(d_{limite} - d, 0)$, onde d_{limite} é a distância máxima onde o efeito do obstáculo impacta no robô e d a distância até o obstáculo mais próximo. Por padrão, as análises realizadas neste projeto consideram $d_{limite} = 100$.

Após serem definidos os pesos para cada nó, é gerado um grafo direcionado, onde cada aresta (a, b) definida para o caso não ponderado é dividida em duas arestas direcionadas (a, b) e (b, a). O peso de cada aresta fica definido como o custo associado ao nó em que ela incide: o peso da aresta (a, b) equivale ao valor do peso de b, enquanto o peso da aresta (b, a) é equivalente ao peso de a.

Os algoritmos 9, 10, 11 e 12 representam os pseudocódigos dos elementos participantes na criação do grafo ponderado, onde os algoritmos 9, 10 e 12 assumem os mesmos papéis dos algoritmos 6, 7 e 8, respectivamente, no caso não ponderado; e o Algoritmo 11 é o responsável direto pelo cálculo dos pesos dos nós. A partir da definição dos nós do grafo, o Algoritmo 11 calcula de maneira iterativa o peso de cada nó com base na sua distância ao obstáculo mais próximo e no tamanho do veículo, com um limite de 100 iterações por nó.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Algorithm 9 $CREATE_TREE(img, transitable_color, size)$

```
(h, l, _) ← dimensões de img
node_list ← lista vazia
for y ← 0 to h − 1 do
    linha ← lista vazia
    for x ← 0 to l − 1 do
        if IS_TRANSITABLE(x, y, img, transitable_color) then
            adicionar novo Node() em linha
        else
            adicionar NULL em linha
    adicionar linha em node_list
for y ← 0 to h − 1 do
    for x ← 0 to l − 1 do
        if node_list[y][x] ≠ NULL then
            node_list[y][x].w ← CHECK_NODE(node_list, h, l, x, y, size)
return node_list
```

Algorithm 10 $IS_TRANSITABLE(x, y, img, transitable_color)$

```
cor ← img[y][x]
if cor = transitable_color then
    return TRUE
else
    return FALSE
```

Algorithm 11 $CHECK_NODE(node_list, h, l, x, y, size)$

```
limit ← 100
for i ← 1 to limit − 1 do
    for cada movimento (dx, dy) em movements do
        mov_x ← i · dx
        mov_y ← i · dy
        nx ← x + mov_x
        ny ← y + mov_y
        if dx = 0 ou dy = 0 then
            d ← i · 1.0
        else
            d ← i · √2
        if (ny < 0 ou ny ≥ h ou nx < 0 ou nx ≥ l) ou node_list[ny][nx] = NULL
        then
            if d ≤ size then
                return INF {nó não é transitável}
            else
                weight ← limit − i
                return weight
return 0
```

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Algorithm 12 Classe *Node*

Classe *Node*:
    $parent_x \leftarrow -1$
    $parent_y \leftarrow -1$
    $w \leftarrow 0$ {peso do nó}
Construtor: inicializa os atributos acima

Na Figura 6, apresentamos um exemplo simples para visualização do processamento de uma imagem segmentada para sua representação em forma de grafo. A primeira imagem representa a imagem segmentada de um terreno, onde a cor verde equivale às áreas transitáveis e o restante corresponde às áreas não transitáveis. Na segunda imagem, temos uma divisão em grade da primeira imagem, onde cada quadrado corresponde a um pixel. A terceira imagem representa a aplicação da máscara binária na segunda imagem, onde os quadrados brancos são transitáveis e os quadrados pretos são não transitáveis. Assim, para cada pixel transitável, adicionamos um nó ao grafo. Disso, obtemos a quarta imagem, onde os pixels em branco correspondem aos nós do grafo e são conectados por arestas aos nós adjacentes, levando em consideração o movimento em 8 direções. A quinta imagem representa o grafo com peso nos nós, refletindo suas proximidades a um obstáculo. A sexta imagem representa o estado final do grafo ponderado, com as arestas direcionadas possuindo um peso relativo ao nó nos quais elas incidem (aresta vermelha possui peso 1, azul peso 2).

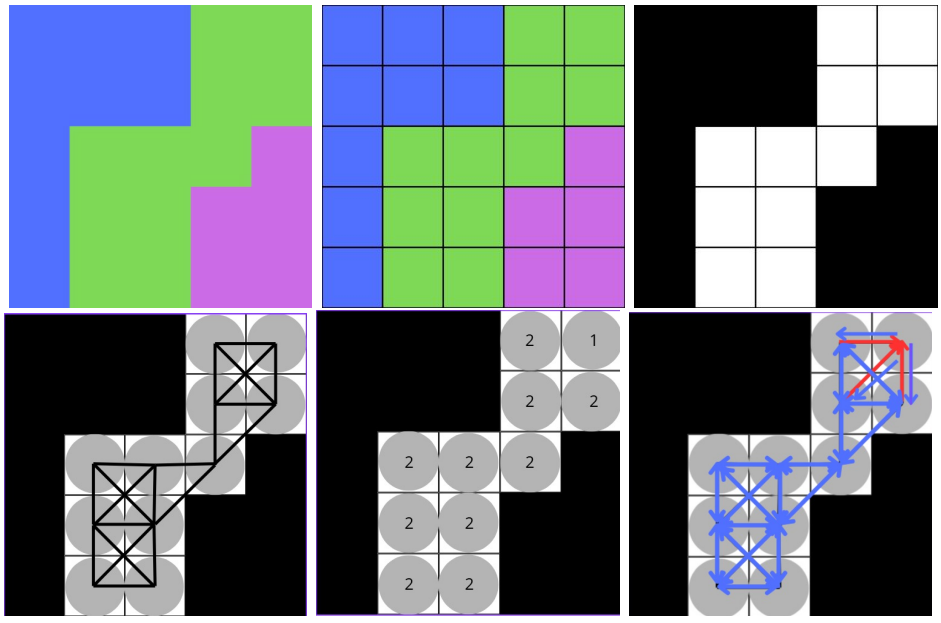


Figura 6 – Imagens exemplificando a criação dos grafos. Da esquerda para a direita, de cima para baixo: imagem segmentada, divisão em pixeis, máscara binária, vértices e arestas, pesos no vértice, arestas direcionadas com peso (vermelho indica peso 1, azul peso 2).

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

4.3 Experimentação e Análise dos resultados

Executamos os algoritmos de Dijkstra, A* e RRT para o planejamento de trajetória nas 5 imagens aéreas apresentadas na figura 7 (1 imagem do CTEx e 4 imagens escolhidas aleatoriamente do dataset utilizado), com suas respectivas imagens segmentadas na figura 8, e as máscaras binárias na figura 9. Para os algoritmos exatos (Dijkstra e A*), calculamos os caminhos nos grafos ponderados e não ponderados e comparamos os tempos de execução e distâncias encontrados. Para o RRT, como ele atua diretamente na imagem e não está associado a nenhum grafo, tomamos apenas um resultado (o melhor dentre 5 tentativas) para cada imagem; e, para a sua execução, definimos um limite máximo de 5.000 iterações e um tempo limite de 2 minutos em cada tentativa. Em seguida, consolidamos os resultados e analisamos a viabilidade de cada abordagem.



Figura 7 – As 5 imagens aéreas utilizadas para as análises (Imagens 1 a 5, da esquerda para a direita, de cima para baixo)

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

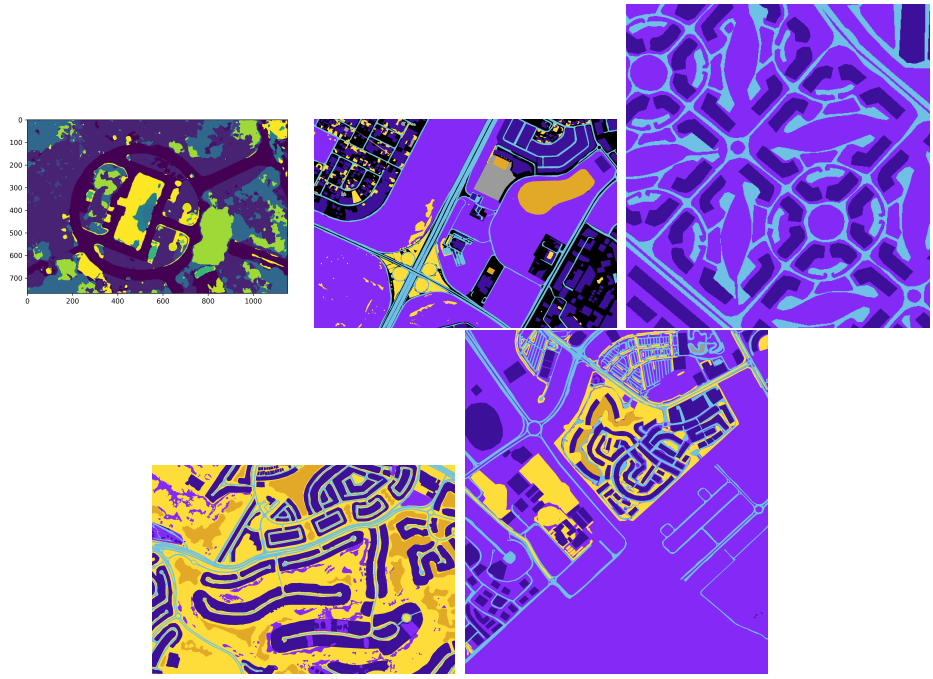


Figura 8 – As correspondentes imagens segmentadas. Na primeira imagem, a cor roxo escuro representa as áreas transitáveis, enquanto, no restante, a cor azul possui essa função.



Figura 9 – Imagens com aplicação da máscara binária.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

5 RESULTADOS E DISCUSSÃO

Neste capítulo, analisa-se o desempenho dos algoritmos A*, Dijkstra e RRT usando como entrada os grafos com e sem ponderação apresentados no capítulo anterior. As Tabelas 2, 3 e 4 apresentam os resultados obtidos para as 5 imagens apresentadas anteriormente com os algoritmos A*, Dijkstra e RRT, respectivamente. No caso do algoritmo RRT, não utiliza grafos, porém A* e Dijkstra foram analisados para os 2 tipos de grafos construídos a partir das imagens, com ponderação e sem ponderação, em função de tempo de execução do algoritmo e distância do caminho obtido.

Imagem	**Sem ponderação**		**Com ponderação**	
	Distância	**Tempo (s)**	**Distância**	**Tempo (s)**
Imagem 1	983.25	1.77	1088.99	1.85
Imagem 2	2652.87	1.97	2831.26	1.94
Imagem 3	435.40	0.26	435.40	0.23
Imagem 4	3078.18	1.76	3894.64	1.98
Imagem 5	2573.96	2.32	2857.82	2.15

Tabela 2 – Resultados do algoritmo A*

Imagem	**Sem ponderação**		**Com ponderação**	
	Distância	**Tempo (s)**	**Distância**	**Tempo (s)**
Imagem 1	983.25	1.93	1093.06	2.04
Imagem 2	2652.87	1.90	2837.40	1.94
Imagem 3	435.40	0.32	448.70	0.28
Imagem 4	3078.18	1.82	3905.76	1.89
Imagem 5	2573.96	2.34	2672.88	2.45

Tabela 3 – Resultados do algoritmo de Dijkstra

Analisando as tabelas para o desempenho dos algoritmos A* e Dijkstra, não foram encontradas diferenças substanciais quanto à distância, com ambos os algoritmos tendo o mesmo resultado para o caso não ponderado e resultados muito próximos para o caso ponderado. O algoritmo de Dijkstra apresenta tempos de execução levemente piores do que o de A*, porém este resultado já era esperado dada a característica do A* de explorar o menor número necessário de nós. Assim, infere-se que o A* é mais adequado para o problema de planejamento de trajetórias proposto, pois apresenta uma maior velocidade

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

Imagem	**Distância**	**Tempo (s)**
Imagem 1	1830.85	1.95
Imagem 2	N/A	Tempo Limite
Imagem 3	459.09	0.77
Imagem 4	N/A	Tempo Limite
Imagem 5	2952.53	73.11

Tabela 4 – Resultados do algoritmo RRT

no cálculo do caminho. Nas Figuras 10, 11 e 12, são apresentados os caminhos encontrados para os algoritmos A*, Dijkstra e RRT, respectivamente.

Na Figura 10, os caminhos em azul claro representam a aplicação sobre o grafo sem peso; e, em azul escuro, sobre o grafo ponderado.

Na Figura 11, os caminhos em verde claro representam a aplicação sobre o grafo sem peso; e, em verde escuro, sobre o grafo ponderado.

Na Figura 12, cada ponto vermelho é 1 passo do algoritmo; a linha azul representa o caminho escolhido, se encontrado dentro do limite de tempo.

Nas Figuras 10, 11 e 12, os círculos vermelhos representam os pontos inicial e final dos caminhos.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

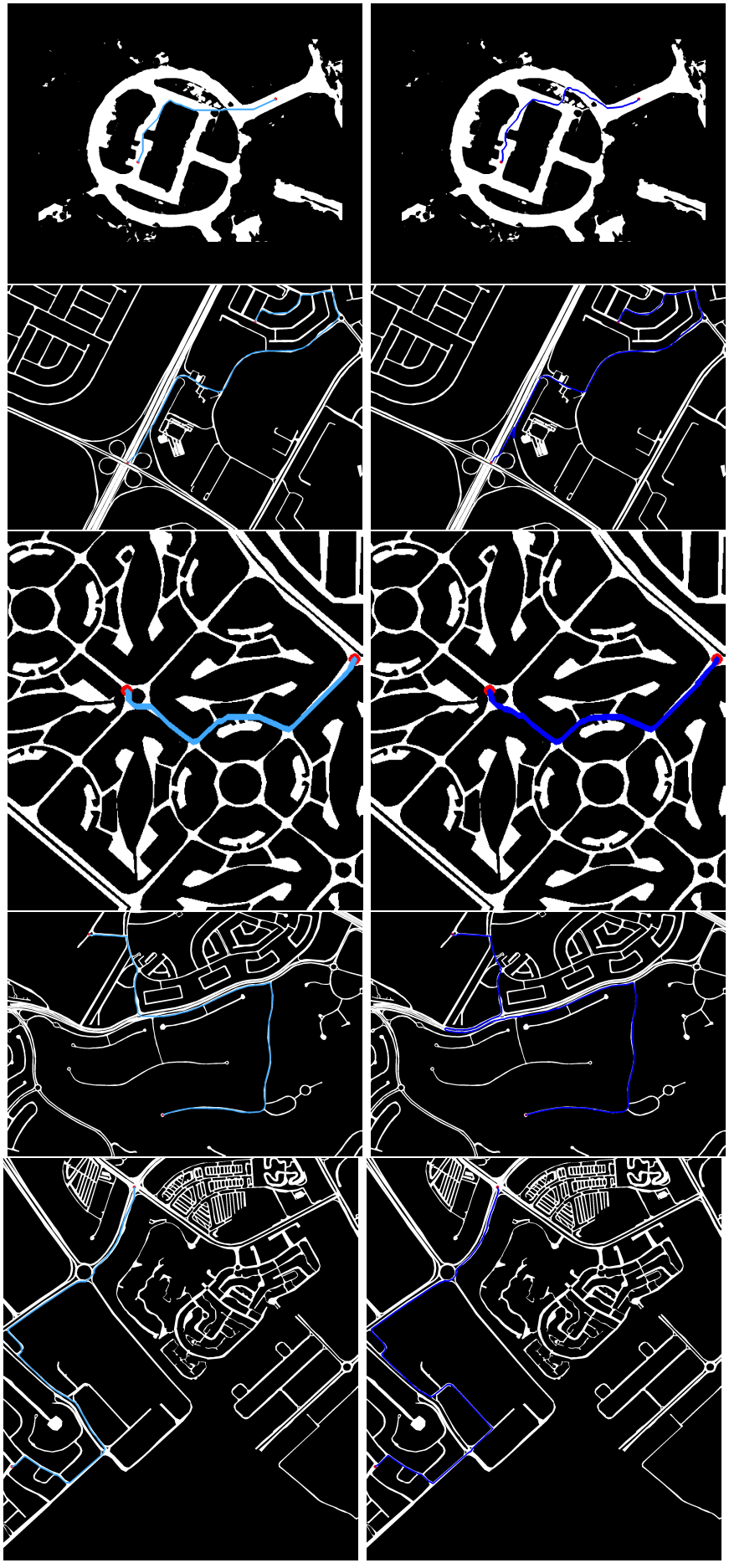


Figura 10 – Caminhos encontrados ao aplicar o A* nas imagens

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

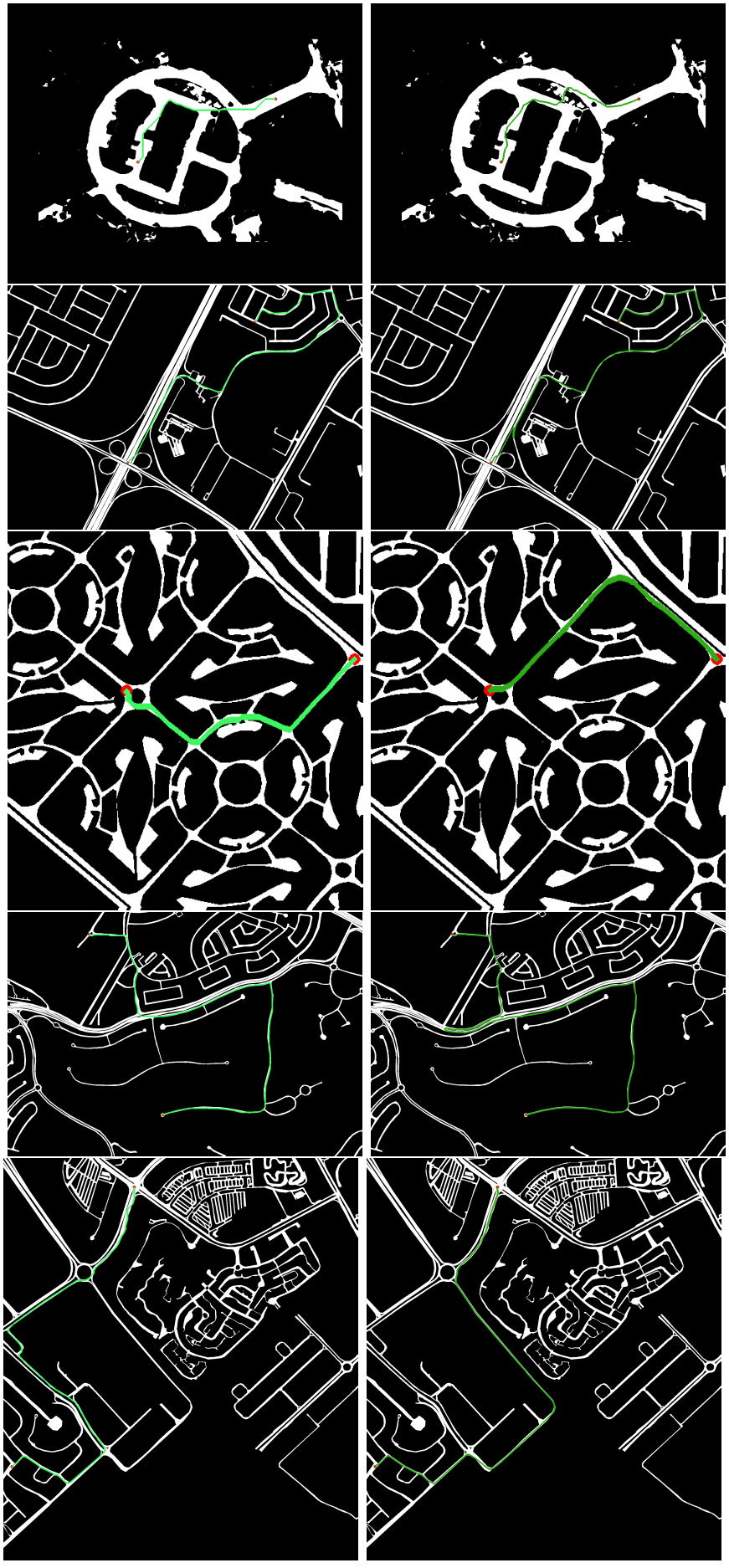


Figura 11 – Caminhos encontrados ao aplicar o algoritmo de Dijkstra nas imagens

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

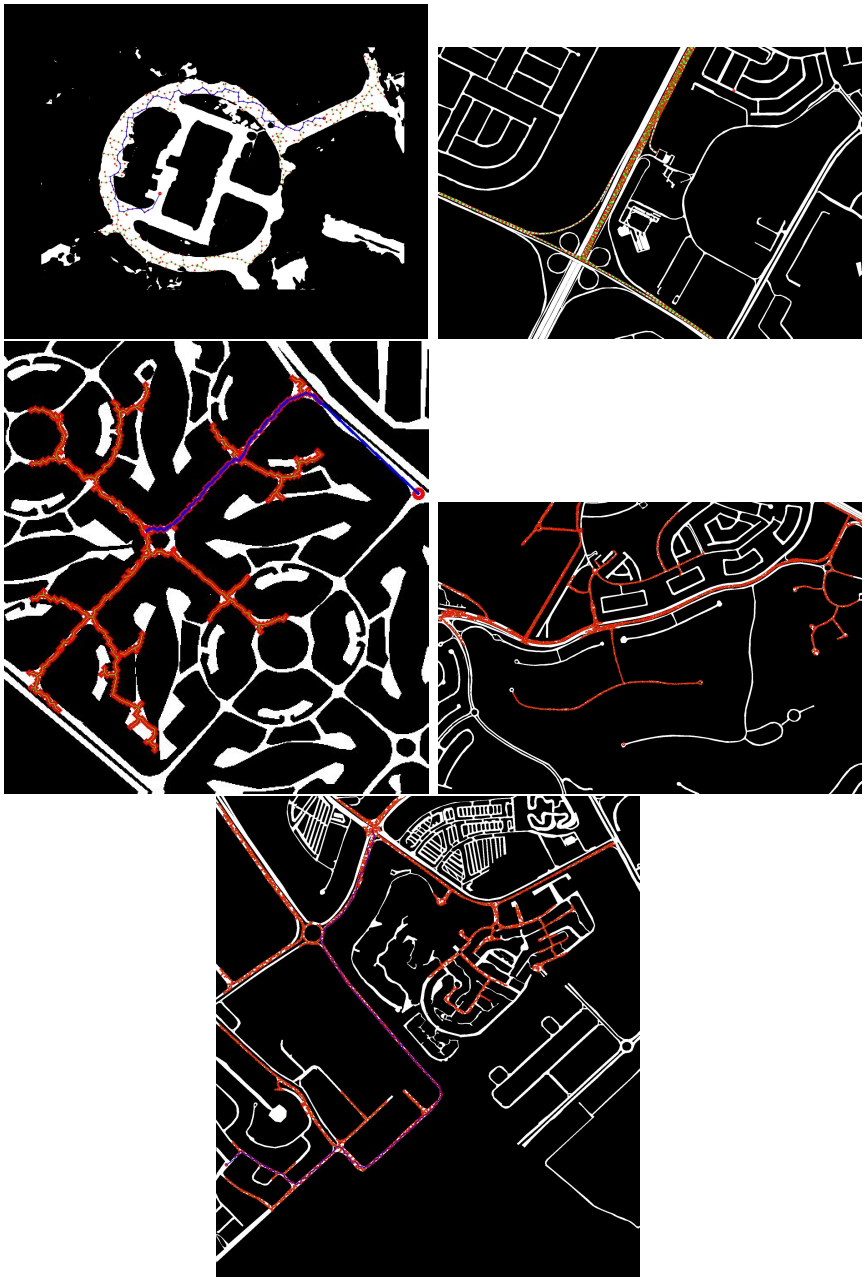


Figura 12 – Caminhos encontrados ao aplicar o RRT nas imagens

Analisando os caminhos gerados, é evidente que os algoritmos de Dijkstra e A*, quando aplicados em grafos não ponderados, tendem a aproximar os pontos do caminho aos obstáculos. Esse comportamento é esperado, pois o algoritmo sempre busca a menor distância. Entretanto, ele desconsidera o risco de praticar tais manobras e coloca o veículo em situações de risco, ou posições onde o trânsito seria inviável. Para o caso ponderado, vemos que os caminhos obtidos se distanciam dos obstáculos e, por esse motivo, possuem comprimentos maiores que no caso não ponderado. No entanto, esse resultado seria mais realista para um sistema de navegação.

Analisando os caminhos obtidos e as informações da tabela, podemos perceber que

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

o caso para os grafos ponderados percorre uma distância maior para todas as iterações. Isto é reflexo do fato que os pesos foram inseridos com o propósito de diminuir a proximidade do robô com os obstáculos, e comprova a viabilidade do mesmo para cumprir o proposto. Em relação ao tempo de execução dos algoritmos, não foi observada grande diferença entre as abordagens. Assim, a inserção dos pesos no grafo não impacta no desempenho do sistema, e deve, então, ter precedência sobre a abordagem não ponderada, uma vez que seus resultados são mais adequados ao caso geral de planejamento de trajetórias.

Vale ressaltar que, na maioria das aplicações reais, o veículo terrestre não deve ser considerado adimensional, pois suas características físicas implicam diretamente na sua capacidade de navegação do terreno. Utilizar-se do caso não ponderado é permitir uma possível colisão do UGV com um obstáculo qualquer, possivelmente danificando-o e impactando no seu desempenho.

Ao analisarmos o algoritmo RRT, temos um resultado diferente do esperado. Pela falta de necessidade de utilização de grafos e pela sua simplicidade computacional, esperávamos que ele apresentasse resultados mais rapidamente que os algoritmos baseados em grafo, mesmo retornando um caminho não ótimo. Entretanto, isso não foi observado e, para todos os exemplos aplicados, tivemos resultados de tempo piores para o RRT do que para ambos algoritmos de Dijkstra e A*.

Analisando a Figura 12, podemos relacionar esse resultado à amplitude das imagens, e a proporção entre áreas transitáveis e não transitáveis. Tomando como exemplo imagens 3 e 5, as quais abrangem uma maior área e suas poucas zonas transitáveis são muito estreitas, o RRT apresentou dificuldade para encontrar um caminho e excedeu o tempo limite definido, pois a maioria dos passos eram dados em direções não favoráveis e tivemos um comportamento não esperado. No entanto, podemos perceber que para as imagens 1 e 3, ele é capaz de encontrar um caminho em tempo hábil, até mesmo com distâncias similares aos outros algoritmos.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

6 CONCLUSÃO

O presente trabalho explorou tópicos altamente relevantes ao problema de planejamento de trajetória para veículos terrestres não tripulados. Nele, contextualizou-se o desafio de planejamento de caminhos em um contexto de colaboração entre drones aéreos e terrestres para o percurso em terreno não mapeado e como o planejamento de caminhos deve ser abordado.

A revisão da literatura aqui realizada permite entender a distribuição e evolução das pesquisas relacionadas ao tema no contexto mundial. Também foram apresentados o estado da arte das soluções para o problema de navegação autônoma em sistemas embarcados.

Propõe-se uma nova abordagem baseada em grafos para o planejamento de caminhos sobre imagens aéreas estáticas. Esta abordagem permite o desenvolvimento de soluções exatas e com comportamento determinístico, características desejáveis em diferentes aplicações. Também são feitas considerações sobre as características físicas de um sistema robótico e suas limitações de mobilidade, generalizando o modelo proposto.

Ainda, os algoritmos propostos (Dijkstra e A*) são comparados entre si e com o algoritmo não-determinístico Rapidly-exploring Random Tree (RRT), solução proposta em (1). Esta comparação permite um maior entendimento das vantagens e desvantagens de cada um, e instiga o desenvolvimento de novas soluções

Com isso, criam-se novas possibilidades para implementação de planejamento de trajetória no projeto SVTRP (Sistemas de Veículos Terrestres Remotamente Pilotados) do Centro de Tecnologia do Exército (CTEx), e aumenta a disponibilidade de soluções para o problema em questão.

6.1 Perspectivas Futuras

Apesar do presente projeto cumprir os objetivos propostos, algumas questões importantes não foram abordadas por não se encaixarem no escopo do mesmo. Esperamos que este trabalho possa servir de inspiração para outros, por isso, nesta seção trazemos uma breve análise de alguns pontos relevantes para estudos futuros.

Primeiramente, a solução aqui apresentada é adaptada para o caso de um ambiente estático. Isso abre o questionamento de como devemos lidar com incerteza nas imagens e características dinâmicas do ambiente, como o surgimento de novos obstáculos durante o percurso. Existe uma grande área de pesquisa que visa resolver estes problemas, utilizando de algoritmos não determinísticos abertos à modificação após definição do trajeto, como o D*. Este é um problema altamente complexo, com extrema relevância em aplicações do

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

mundo real.

Outra hipótese que pode ser feita em cima dos resultados aqui obtidos é o impacto que o uso dos grafos pode ter em uma situação de composição de diversas imagens mais próximas ao solo e em tempo real. Como observado na Tabela 4 e na Figura 12, é possível inferir que o RRT é melhor aplicado em situações em que a imagem do terreno possui maior área transitável. Assim, para uma situação de percurso dinâmico, onde o sistema recebe uma sequencia e imagens do terreno e precisa calcular as rotas rapidamente, algoritmos baseados em grafo possivelmente apresentariam desempenho inferior. Nesse caso, talvez algoritmos aplicados diretamente na imagem, como o RRT executem mais rapidamente, por excluírem o passo de criação dos grafos. Um estudo comparando os desempenhos em uma situação similar a descrita é desejável, para desenvolvimento de soluções mais responsivas.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

REFERÊNCIAS

1 JUNIOR, F. L. *Cooperação entre Plataformas Robóticas Terrestres e Aéreas para aquisição, rastreamento e perseguição de alvos, utilizando Visão Computacional e Inteligência Artificial*. Tese (Doutorado) — Instituto Militar de Engenharia, Rio de Janeiro, 2024.

2 FINN, A.; SCHEDING, S. Developments and challenges for autonomous unmanned vehicles. *Intelligent Systems Reference Library*, Springer, v. 3, p. 128–154, 2010.

3 MADRIDANO, Á.; AL-KAFF, A.; MARTÍN, D.; ESCALERA, A. D. L. Trajectory planning for multi-robot systems: Methods and applications. *Expert Systems with Applications*, Elsevier, v. 173, p. 114660, 2021.

4 CHEN, X.; ZHAO, M.; YIN, L. Dynamic path planning of the uav avoiding static and moving obstacles. *Journal of Intelligent & Robotic Systems*, Springer, v. 99, p. 909–931, 2020.

5 CHEN, W.-K. *Graph theory and its engineering applications*. [S.l.]: World Scientific, 1997. v. 5.

6 DEO, N. *Graph theory with applications to engineering and computer science*. [S.l.]: Courier Dover Publications, 2016.

7 BEAMER, S.; ASANOVIĆ, K.; PATTERSON, D. Direction-optimizing breadth-first search. *Scientific Programming*, Wiley Online Library, v. 21, n. 3-4, p. 137–148, 2013.

8 DIJKSTRA, E. A note on two problems in connexion with graphs. *Numerische Mathematik*, v. 1, p. 269–271, 1959.

9 HART, P. E.; NILSSON, N. J.; RAPHAEL, B. A formal basis for the heuristic determination of minimum cost paths. *IEEE transactions on Systems Science and Cybernetics*, IEEE, v. 4, n. 2, p. 100–107, 1968.

10 XU, T. Recent advances in rapidly-exploring random tree: A review. *Heliyon*, Elsevier, 2024.

11 WU, Y.; WU, S.; HU, X. Cooperative path planning of uavs & ugvs for a persistent surveillance task in urban environments. *IEEE Internet of Things Journal*, IEEE, v. 8, n. 6, p. 4906–4919, 2020.

12 LI, J.; DENG, G.; LUO, C.; LIN, Q.; YAN, Q.; MING, Z. A hybrid path planning method in unmanned air/ground vehicle (uav/ugv) cooperative systems. *IEEE Transactions on Vehicular Technology*, IEEE, v. 65, n. 12, p. 9585–9596, 2016.

13 ROPERO, F.; MUÑOZ, P.; R-MORENO, M. D. Terra: A path planning algorithm for cooperative ugv–uav exploration. *Engineering Applications of Artificial Intelligence*, Elsevier, v. 78, p. 260–272, 2019.

14 ZHOU, X.; YU, X.; ZHANG, Y.; LUO, Y.; PENG, X. Trajectory planning and tracking strategy applied to an unmanned ground vehicle in the presence of obstacles. *IEEE Transactions on Automation Science and Engineering*, IEEE, v. 18, n. 4, p. 1575–1589, 2020.

Docusign Envelope ID: 8F3E4BF3-3990-412A-880C-7E297DC46EAB

15 LIU, J.; ANAVATTI, S.; GARRATT, M.; ABBASS, H. A. Modified continuous ant colony optimisation for multiple unmanned ground vehicle path planning. *Expert Systems with Applications*, Elsevier, v. 196, p. 116605, 2022.

16 THORESEN, M.; NIELSEN, N. H.; MATHIASSEN, K.; PETTERSEN, K. Y. Path planning for ugvs based on traversability hybrid a. *IEEE Robotics and Automation Letters*, IEEE, v. 6, n. 2, p. 1216–1223, 2021.

17 LOOP, H. in the. *Semantic segmentation of aerial imagery*. Disponível em: <https://www.kaggle.com/datasets/humansintheloop/semantic-segmentation-of-aerial-imagery>. Acesso em: 28 sep 2025.